

SUPER ADMIN PORTAL · MEDIA DEPARTMENT

Media Portal — Project-Based Marketing ERP

Complete functional and logic specification: how every project runs its own marketing lifecycle — planning, execution, approval, publishing, performance, and reporting — inside one unified Super Admin Portal.

Document type: Functional & Logic Specification

Scope: Media Portal (all projects, all roles)

Prepared: 2026-07-05

Companion document: Phased Implementation Roadmap (Appendix A)



Table of Contents

Part I — Architecture & Flow

1. Vision & Architecture
2. Main Business Flow
3. Project Logic (Data Isolation)

Part II — Operational Logic

4. Campaign Logic
5. Content Workflow
6. Task Logic
7. Weekly Planning Logic
8. Checklist Logic

Part III — Financial & Performance Logic

9. Budget Logic
10. KPI Logic
11. Marketing Funnel Logic

Part IV — Governance & Operations Logic

12. Approval Logic
13. Calendar Logic
14. Team Logic
15. Notification Logic

Part V — Intelligence Layer

16. Report Logic
17. Analytics Logic
18. Dashboard Logic

Part VI — Access & Structure

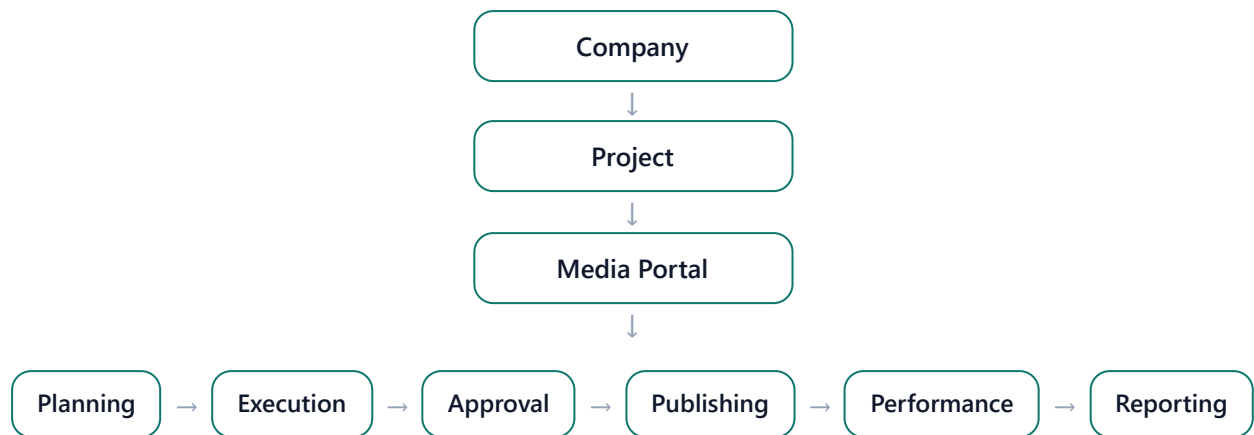
19. Role-Based Logic
20. Backend Relationship Logic
21. Overall Business Logic

Appendices

- A. Implementation Status & Roadmap
- B. Data Model Reference

Vision & Architecture

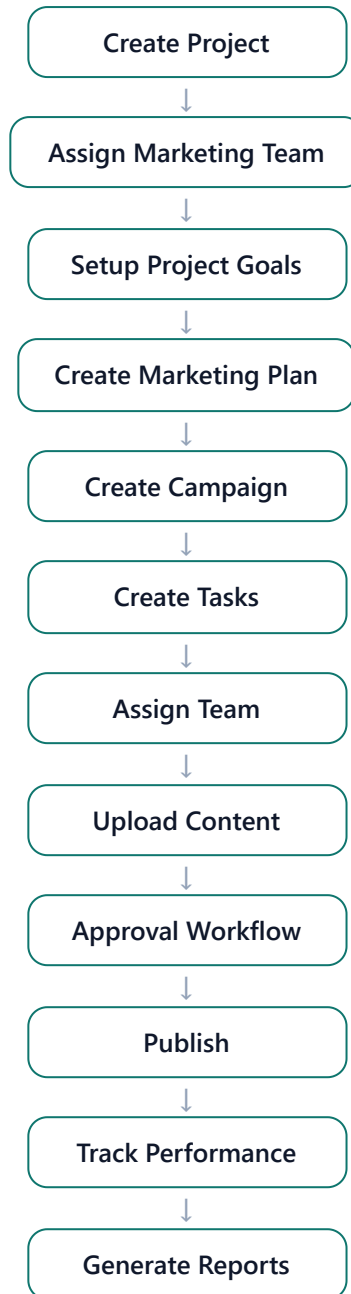
The Media Portal is not a single shared workspace — it is a **project-based marketing ERP**. Every company project gets its own self-contained marketing ecosystem (goals, campaigns, budget, content, tasks, reports, analytics, team, assets), while every project is still managed, monitored, and governed from one Super Admin Portal.



Current state: the shared, portal-wide Project model is the tenant boundary. Media data (Campaign, Media/content, Task) attaches to it via `projectId`, and every `list/create/update/delete` query in the Media module is scoped by that id — this is what keeps Planning→Reporting isolated per project.

Main Business Flow

This is the master lifecycle every project's marketing effort follows, end to end:



Project Logic — Data Isolation

Every project maintains its own Marketing Goals, Campaigns, Budget, Content, Tasks, Reports, Analytics, Team, and Assets. **No data mixes between projects.**

Project A

- Campaigns
- Budget
- Reports

Project B

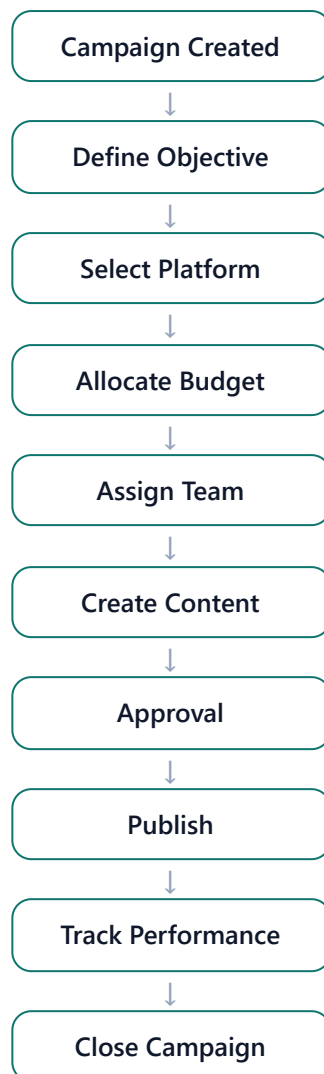
- Campaigns
- Budget
- Reports

Project C

- Campaigns
- Budget
- Reports

Enforcement mechanism: `backend/middlewares/project.middleware.js` — `requireProjectContext` (mutations) and `attachOptionalProjectContext` (reads) extract `projectId` from query/header/body/params, check it against the user's assigned project list, and every service query filters by it. This is the single invariant every other section in this document depends on.

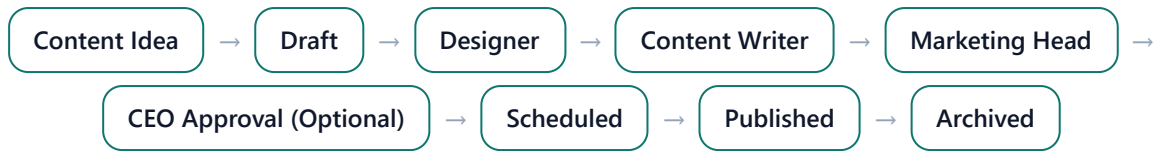
Campaign Logic



Implemented (Phase 1): Campaign model (`backend/models/department/Campaign.js`) with a status enum matching this exact lifecycle (draft → objective-defined → platform-selected → budget-allocated → team-assigned → content-in-progress → in-approval → scheduled → live → tracking → closed), full CRUD at `/api/dept/media/campaigns`.

Planned (Phase 2): a validated stage-transition state machine (`advanceLifecycleStage`) so a campaign can only move forward through this exact sequence, plus a `lifecycleHistory[]` audit trail recording who moved it and when.

Content Workflow



Every stage keeps history — no step silently overwrites the previous one.

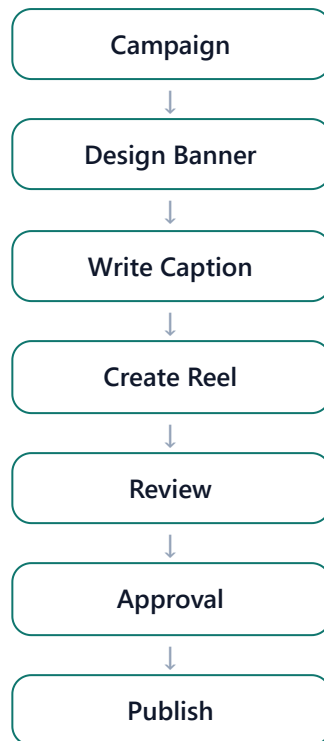
Reused as-is: the generic, multi-step ApprovalWorkflow engine

(`backend/services/approvalEngine.service.js`) already supports arbitrary role-ordered steps with per-step `status/decidedBy/decidedAt/remarks` — exactly the history requirement above. It's already wired into Media asset/content approval requests.

Planned (Phase 2): replace the current generic step list (`media → team_lead → department_head → marketing_head → client_viewer → admin`) with `[graphic_designer, content_writer, marketing_head, {ceo, optional:true}]`, and add `scheduled→published→archived` as post-approval lifecycle states on the content record itself.

Task Logic

Each campaign automatically generates its own operational tasks. Example:



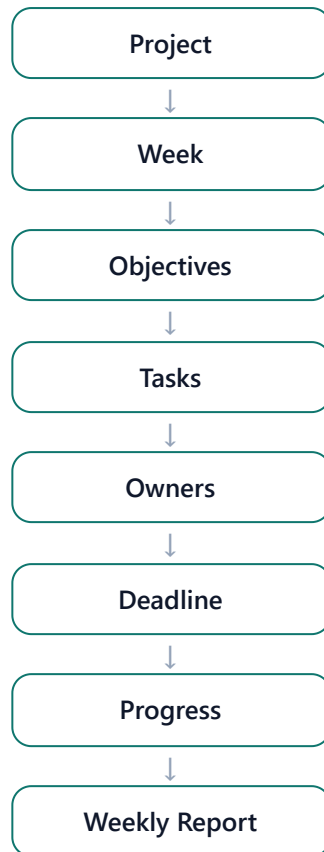
Task Status



Reused as-is: the generic Task model (`backend/models/common/Task.js`) already covers assignment, due dates, dependencies, and an auto-computed overdue flag. Its native status enum (`pending/in-progress/review/completed/cancelled`) maps onto the labels above at the UI layer (`in-progress` displays as "Working") to avoid a schema migration.

Planned (Phase 3): a `campaignTask.service.js` that auto-generates the task set above whenever a campaign enters the `content-in-progress` stage, plus a `campaignId` field added to Task so tasks trace back to their originating campaign.

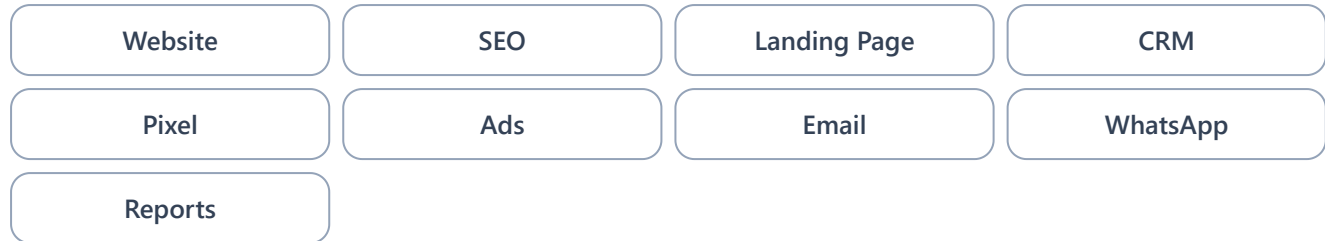
Weekly Planning Logic



Planned (Phase 6): new `WeeklyPlan` model (`projectId`, `weekStart/End`, `objectives[{{text, linkedTaskIds[], ownerId, status}}]`) linking each week's objectives to real `Task` records so progress rolls up automatically instead of being tracked manually.

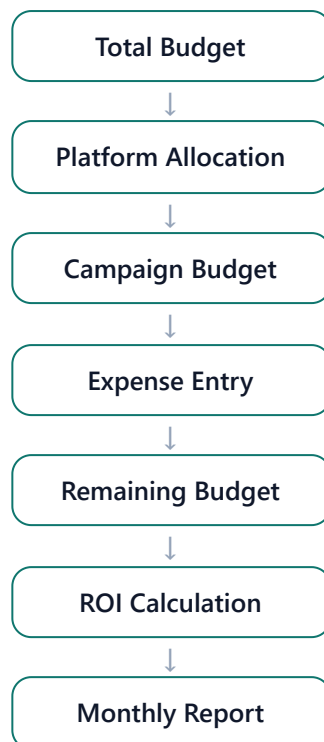
Checklist Logic

Each project carries a predefined set of operational checklists:



Planned (Phase 6): new `ProjectChecklist` model (`projectId`, `checklistType` enum matching the nine types above, `items[{{label, done, completedBy, completedAt}}]`) with `completionPercent` computed on save — mirrors the existing `Project.progress` field pattern already in `common/Project.js`.

Budget Logic



Auto-calculated formulas

Remaining Budget = Total Budget - Used Budget

ROI (%) = ((Revenue - Spend) / Spend) × 100

ROAS = Revenue / Spend

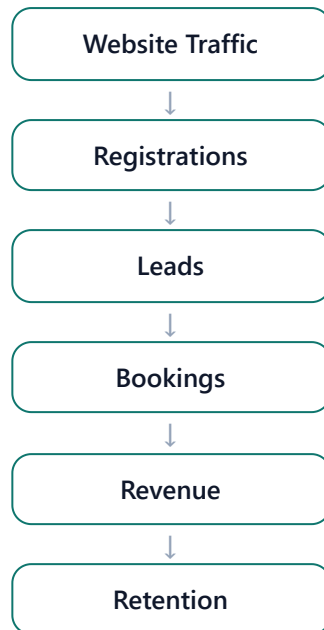
Cost Per Lead (CPL) = Spend / Number of Leads

Implemented (Phase 1): `Media.budgetImpact{spend, roiAtSnapshot}` replaces the old `unschema'd` metadata fields, and the dashboard overview aggregates real spend/ROI from it for advertisement records.

Planned (Phase 4): dedicated `BudgetAllocation` model (total → platform → campaign breakdown), a media-scoped expense ledger, and a pure `budgetCalc.service.js` implementing the four formulas above as testable functions independent of any UI.

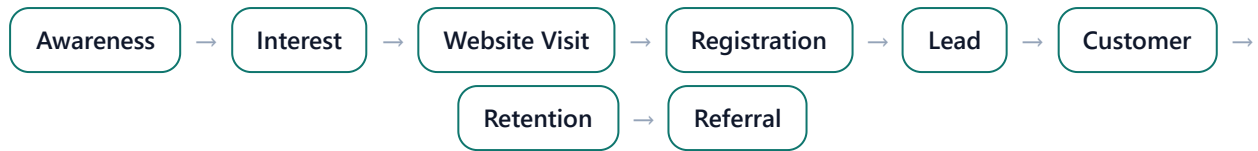
KPI Logic

Every KPI updates automatically as underlying activity happens — the dashboard always reads the latest values, never a stale cache.



Planned (Phase 4): new KpiSnapshot model (projectId, period, websiteTraffic, registrations, leads, bookings, revenue, retention) plus kpiFunnel.service.js: computeConversionFunnel(). Persisted as period snapshots (following the existing performanceSystem.service.js upsert pattern) rather than recomputed live every request, for speed.

Marketing Funnel Logic



Every stage stores its own count so conversion between any two adjacent stages can be computed:

$$\text{Conversion \%} = (\text{Current Stage Count} / \text{Previous Stage Count}) \times 100$$

This funnel and the KPI funnel in Section 10 share the same underlying KpiSnapshot data — the funnel is simply that data read stage-by-stage with the conversion formula applied between adjacent stages.

Approval Logic



Rejection path



Reused as-is: ApprovalWorkflow model + `approvalEngine.service.js`

(`createApprovalRequest/decideApprovalRequest`) already implement exactly this approve/reject/resubmit cycle, keyed generically by `module/entityType/entityId`. No new engine needed — only the step-role list changes per Section 5.

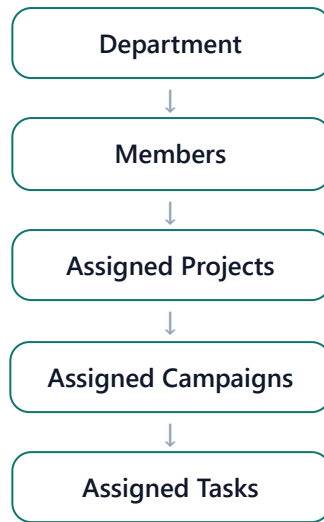
Calendar Logic

Everything with a date attached surfaces automatically on one calendar — nothing is entered twice.



Planned (Phase 5): `calendar.service.js: getAggregatedCalendar(projectId, range)` — a live parallel query across Campaign (start/end/publish dates), Task (due dates), Media (publish/approval dates), and Budget (period boundaries), normalized into one event list. Starting as a live aggregation rather than a separate materialized collection, revisited only if performance requires it.

Team Logic



Each user sees only assigned work — this is the same project-scoping invariant from Section 3, applied at the individual-user level via their project/campaign/task assignments.

Reused as-is: the generic Team model (`backend/models/manager/Team.js`) already models `Department → Manager → Members[{employee, role}]`.

Notification Logic

Trigger-based — each of the following fires a notification the moment its condition is met:

Task Assigned

Task Completed

Approval Pending

Campaign Ending

Budget Exceeded

Deadline Tomorrow

Report Ready

Socket.IO is already running in the backend (currently emitting only presence events) — the transport for real-time delivery exists.

Planned (Phase 5): event-fired triggers (Task Assigned/Completed, Approval Pending) call a new `notificationTrigger.service.js` directly from existing mutation points — no new infrastructure needed. Time-based triggers (Campaign Ending, Budget Exceeded, Deadline Tomorrow, Report Ready) require introducing a scheduler (node-cron vs. agenda — an open decision) since none exists in the codebase today.

Report Logic

Reports are generated from live data, never hand-assembled:

Tasks + Campaign + Budget + Analytics + KPI + Content = Weekly Report

The same formula produces Monthly, Quarterly, and Annual reports — only the period window changes.

Planned (Phase 8): MarketingReport model persisting a dashboard-aggregate snapshot per period, generated on-demand or automatically via the Phase 5 scheduler, feeding the "Report Ready" notification trigger.

Analytics Logic

Collect

Website	Social Media	Ads	Leads
Revenue	Conversions		

Display cadence

Daily	Weekly	Monthly	Yearly
-------	--------	---------	--------

Dashboard Logic

The dashboard is a pure aggregation layer — it computes nothing on its own, it reads everything else.

Fetches

Projects	Campaigns	Tasks	Budget
Content	Analytics	KPI	Notifications
Approvals			

Then calculates

Total Projects	Running Campaigns	Completed Tasks	Pending Tasks
Total Budget	Budget Used	Revenue	ROAS
Leads	Conversion	Pending Approval	Upcoming Deadlines

Implemented (Phase 1): the Media dashboard already fixed a duplicate-header rendering bug and now correctly aggregates projects/campaigns/content/approvals into KPI tiles, with running-campaign counts sourced from the real Campaign model.

Planned (Phase 7): a single dashboardAggregate.service.js: getProjectDashboard(projectId) fanning out via Promise.all to every source above (including Phase 4's budget/KPI calc functions), following the snapshot-persistence pattern from performanceSystem.service.js rather than live-querying on every page load.

Role-Based Logic

Spec Role	Access	Existing codebase role
Super Admin	Full access	super_admin — exists
CEO	View + Approval	ceo — exists
Marketing Head	Manage Campaigns	marketing_head — exists
Project Manager	Manage Assigned Projects	project_manager — exists
Designer	Design Tasks Only	graphic_designer — exists
Content Writer	Content Only	content_writer — exists
SEO Executive	SEO Only	seo_specialist — exists
Ads Manager	Ads Only	no dedicated role yet — closest: social_media_manager
Viewer	Read Only	client_viewer — exists

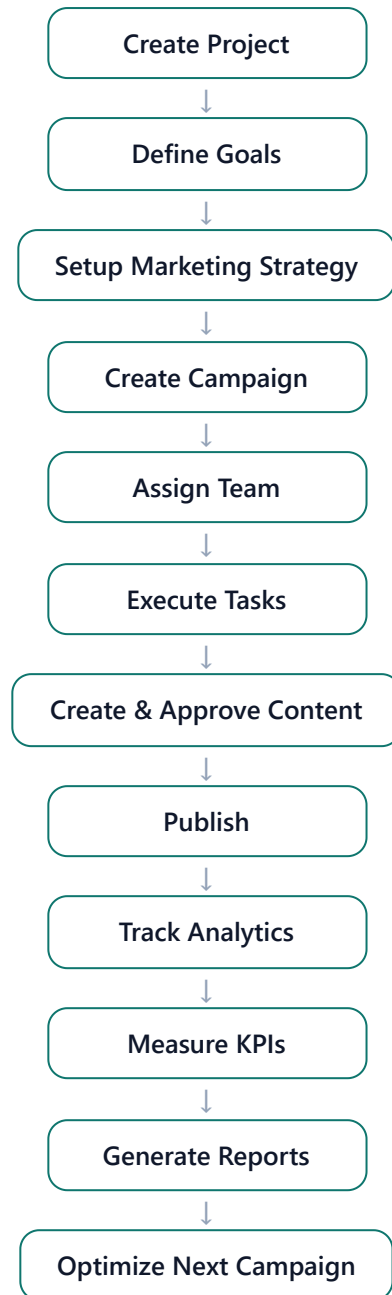
Role strings must be mirrored in three places to take effect: `backend/config/roles.js` (ROLES / ROLE_HIERARCHY / ROLE_PERMISSIONS), `frontend/src/utils/rbac.js` (`baseRoleAccess`), and the portal route guards. Adding the missing "Ads Manager" role is a Phase 0 (cross-cutting) task, done incrementally rather than as one big-bang pass.

Backend Relationship Logic



This tree is the schema-level contract every model in Appendix B follows: everything ultimately traces back to one `Project._id`, directly or through a Campaign that itself belongs to that project.

Overall Business Logic



This is a closed loop by design: the last step feeds back into campaign creation, so every cycle of the Media Portal is informed by the performance data the previous cycle produced — a complete lifecycle management system from planning through continuous optimization.

APPENDIX A

Implementation Status & Roadmap

Full detail lives in the companion Phased Implementation Roadmap. Summary status as of this document's date:

Phase	Scope	Status
0	Role refinement (Ads Manager + per-phase RBAC scoping)	Ongoing
1	Campaign & Goal models, dangling ref fix, campaign CRUD migrated off Media catch-all	Done
2	Campaign lifecycle state machine + redefined content-approval pipeline	Planned
3	Campaign-driven task automation (kanban board)	Planned
4	Budget/ROI/ROAS/CPL + KPI funnel calculation layer	Planned
5	Calendar aggregation + trigger-based notifications (scheduler decision point)	Planned
6	Weekly planning + project checklists	Planned
7	Dashboard aggregation layer	Planned
8	Reports (weekly/monthly/quarterly/annual)	Planned

Dependency order: Phase 1 → 2 → 3 → 4 → (5 and 6 in parallel) → 7 → 8. Phase 0 runs alongside every phase.

Two decisions still open before Phase 5 can start: (1) scheduler library for time-based triggers — node-cron vs. agenda; (2) materialized vs. live aggregation for the Calendar and Dashboard — currently recommended to start live and revisit only if slow.

APPENDIX B

Data Model Reference

Model	Status	Purpose
Project	Existing	Portal-wide tenant boundary; every media record scopes to it.
Campaign	Built — Phase 1	Campaign lifecycle, budget, team, platform, KPI targets.
Goal	Built — Phase 1	Project/campaign-level target tracking (metric, target, current, due date).
Media	Existing (extended)	Assets/content/brand/design/video/social/ads/seo/etc. via <code>moduleType</code> ; now carries a proper <code>budgetImpact</code> field.
Task	Existing	Generic assignment/status/due-date model, reused for campaign task automation.
Team	Existing	Department → manager → members mapping.
ApprovalWorkflow	Existing	Generic multi-step approval engine, reused for content approval.
Budget	Existing	Department/fiscal-year allocation tracking; to gain campaign-level scope.
BudgetAllocation	Planned — Phase 4	Total → platform → campaign budget breakdown.
KpiSnapshot	Planned — Phase 4	Period-based KPI/funnel counts, persisted not live-computed.
CalendarEvent	Planned — Phase 5 (decision pending)	Aggregated dated entities, if a materialized approach is chosen.
WeeklyPlan	Planned — Phase 6	Weekly objectives linked to tasks and owners.
ProjectChecklist	Planned — Phase 6	Per-project operational checklists with computed completion %.
MarketingReport	Planned — Phase 8	Generated period rollups (weekly/monthly/quarterly/annual).